FALL: Prior Failure Detection in Large Scale System Based on Language Model

Jaeyoon Jeong, Insung Baek, Byungwoo Bang, Junyeon Lee, Uiseok Song, and Seoung Bum Kim

***Abstract*—As the scale of high-performance computing systems (HPC) continues to expand, the frequency of failures within the system also increases. The follow-up steps in the event of a failure are necessary because efficient prevention measures are absent. These actions, however, result in decreased operating efficiency as the system normalizes. To solve this problem, recent research has utilized deep learning to predict failures in advance by examining log messages. One common method uses log ID extracted from log messages as input data. However, these methods could lead to information loss in log messages and may not accurately capture log message properties. In this study, we propose the prior failure detection in large-scale systems using language models (FALL) that reflects the properties of log messages. The FALL uses log messages as text data and only uses normal data for training. The FALL makes use of sharpening, which concentrates on the minimal lexical change between normal and abnormal log messages. In addition, by leveraging the characteristics of log messages with limited vocabularies, the FALL utilizes only some tokens for anomaly detection. The experiments' results from log data from real industry-based HPC systems show that the FALL achieves superior performance in early failure detection.**

***Index Terms*—Self-supervised learning, language model, anomaly detection, sharpening, limited vocabulary, HPC system.**

I. Introduction

A HIGH-PERFORMANCE computing (HPC) system is a powerful computational platform that is capable of quickly processing complex calculations [1]. It is usually composed of high-performance hardware and software. In the past, HPC systems were mostly employed for engineering simulations requiring a lot of computing power, as those in nuclear, and particle physics. However, with the advent of artificial intelligence and information technology, HPC systems have become increasingly vital for processing and analyzing large datasets [2], [3]. This has increased demand for HPC systems across different industries, and as a result, HPC system manufacturers are creating more complex and substantial systems. However, because of their complexity and substantial number of parts, HPC systems scale significantly in size while also increasing the incidence of system breakdowns [4], [5].

HPC systems are prone to a wide range of failures, which can lead to the shutdown of the system in question if the failure is severe. The system that was shut down must be restarted and its burden transferred to another system as part of the subsequent step needed to get things back to normal [6], [7]. However, during the process of restoring normal operation to the affected node, the large-scale loading of existing calculations results in a substantial delay, thus reducing the availability of the HPC system. [8], [9]. As a result, the ability to accurately predict system failure is an important aspect because it can improve the availability of the HPC system and reduce the time and cost associated with failures [10]. Utilizing the system log message data produced by the HPC system is an effective way to address this issue.

System log message data is a form of real-time information that reflects the present status of the different components that make up the HPC system, such as nodes [11]. System log message data possesses unstructured characteristics because of its expression in natural language sentences. Furthermore, the collection is unbalanced, with relatively few anomalous log message records compared to regular log message records. In addition, as each component concurrently produces system log messages, a large amount of redundant data is obtained in real-time [12]. System log message data is therefore a prime resource for determining an HPC system's status [11], [12], [13], [14], [15], [16].

The study of the state of HPC systems using system log message data can be broadly divided into two fields: failure detection [11], [15], [16], [17] and failure prediction [12], [14], [18], [19]. Finding out whether some given log message data is normal or abnormal is the goal of the research field of failure detection. This area of study is being actively pursued from different perspectives, utilizing methods such as machine learning [20], [21] and deep learning [11], [15], [16], [17]. The goal of failure prediction, on the other hand, is to identify pre-alerts in the log sequence, a collection of log messages, to anticipate if failures will occur in the HPC system in the future [19]. However, research in this field is more difficult than failure detection because predicting future failures is challenging [12]. Although the log message data was supplied in the form of natural language

Manuscript received 1 April 2023; revised 12 March 2024; accepted 16 March 2024. Date of publication 2 May 2024; date of current version 16 January 2025. This work was supported in part by the Samsung Advanced Institute of Technology, Brain Korea 21 FOUR, in part by the Ministry of Science and ICT (MSIT) in Korea under the ITRC support program supervised, in part by the Institute for Information Communication Technology Planning and Evaluation under Grant IITP-2020-0-01749, and in part by the National Research Foundation of Korea grant funded by the Korea government under Grant RS-2022-00144190. *(Corresponding author: Seoung Bum Kim.)*

Jaeyoon Jeong, Insung Baek, and Seoung Bum Kim are with the School of Industrial and Management Engineering, Korea University, Seoul 02841, South Korea (e-mail: jj950310@korea.ac.kr; insung_baek01@korea.ac.kr; sbkim1@korea.ac.kr).

Byungwoo Bang, Junyeon Lee, and Uiseok Song are with the Samsung Advanced Institute of Technology, Samsung Electronics Company Ltd., Suwon 16678, South Korea (e-mail: byungwoo.bang@samsung.com; junyeon2.lee@samsung.com; us.song@samsung.com).

Digital Object Identifier 10.1109/TDSC.2024.3396166

phrases in earlier studies on failure prediction, the problem was only handled from the standpoint of mapping log messages to log IDs and using them [12], [14], [18]. However, this approach not only results in a loss of information within the log message but also fails to fully consider the unique characteristics of the log message data. This study introduces the failure detection in large-scale systems using language models (FALL), a technique for the early detection of system failures based on natural language processing (NLP). Unlike prior studies on failure prediction, which have mainly relied on mapping log messages to log IDs, this study addresses the problem by applying NLP methods, recognizing that log messages are expressed in natural language sentences. Additionally, this study uses an anomaly score for early failure identification that takes into account the restricted language included in log message data as well as the two separate vocabulary similarities between normal and abnormal log messages. An experiment was performed using data obtained from an actual HPC system to show the effectiveness of the proposed FALL, which achieved the best performance. The main contributions of this study are as follows:

- We present a novel method in the field of early failure detection by applying system log message data as natural language sentences. Unlike previous studies that have mainly used log IDs, this study utilizes the natural language characteristics of log messages to predict failures and enhance performance in different scenarios.
- The proposed method considers the limited vocabulary characteristic of log messages as well as the small vocabulary difference between normal and abnormal log messages. Moreover, a failure prediction strategy for anomaly detection that is well suited to handling imbalanced data is provided for the first time.

The rest of this paper is organized as follows. Section II provides an overview of the related research on natural language anomaly detection and system log-based failure prediction. Section III explains the proposed FALL in detail. Section IV presents the data and experimental setup used in the study as well as the results of the experiment. The study is concluded in Section V by summarizing the key findings and outlining prospective directions for further research.

II. Related Work

A. Anomaly Detection for Text

Obtaining high-quality abnormal data, in reality, can be a time-consuming, and costly task. To address this, the anomaly detection method utilizes only normal data to train the network to identify normal features and effectively classify abnormal and normal instances [22], [23]. This technique has been used for several text classification applications, such as document classification and sentence classification, with the usage of one-class support vector machine (OCSVM) for document classification serving as a representative example [24]. Furthermore, other studies have proposed anomaly detection methods for classifying normal documents and abnormal documents using a clustering algorithm based on latent Dirichlet allocation [25] and a non-negative matrix decomposition algorithm [26]. These approaches are examples of applying anomaly detection techniques to the text field, but they have a limitation in that the embedding is conducted on word units, thus the model may not accurately learn information about the structure and meaning of sentences.

With the development of deep learning, anomaly detection has also gained attention under the name of deep anomaly detection, and various methods have been proposed. Deep anomaly detection allows for the effective learning of complex data which can enhance the performance of the model. Similar investigations using deep anomaly detection in the field of text have also been conducted [27], [28]. As an illustrative example of prior research, [28] proposed the use of an long short-term memory (LSTM)-based autoencoder and showed improvement in the performance of document classification. The model was nevertheless limited in that it could not efficiently learn contextual information because of the vanishing gradient problem inherent to LSTM.

To overcome these limitations, recent studies have suggested anomaly detection methods appropriate for text data [29], [30]. In particular, [30] proposed a representation learning method that combines the replacement masked detection (RMD), a new self-supervised learning technique with efficiently learning an encoder that classifies token replacements accurately (ELECTRA) [31], a language model. They also proposed a text field-appropriate anomaly detection method tailored for sequence data and demonstrated the superior performance of their proposed method using 20 Newsgroups [32] and AGNews [33] datasets. However, the system log message data used in this study differs from the dataset utilized in [30] because it possesses the features of 'similar vocabulary between normal and abnormal' and 'limited vocabulary.' Therefore, this study aims to adapt the method in [30] to suit the characteristics of this system log message data to establish an appropriate method for early failure detection.

B. Failure Prediction in HPC System

Failure prediction is a more difficult field of research than failure detection because it involves predicting future failures that may occur in the system based on the data obtained thus far. Prior research has used a variety of machine learning techniques including decision trees, support vector machines and principal component analysis to anticipate mistakes in HPC systems [21], [34], [35]. Some studies have used deep learning methods to predict system failures [14], [18]. Specifically, [14] used a total of three LSTM layers and proposed an algorithm that predicts failures in the HPC system by assigning an ID to each log message and using it as input to the model. However, the log message itself was not used in these experiments; instead, the log's ID was used, which may have resulted in the loss of message-specific information and a failure to adequately account for the special properties of log message data.

A recent approach utilizing a transformer model was proposed in [12] in which natural language-based preprocessing was applied to system log message data to extract the format of the log messages, which were then assigned an ID and used as input for the model. The model creates a log sequence that is projected to

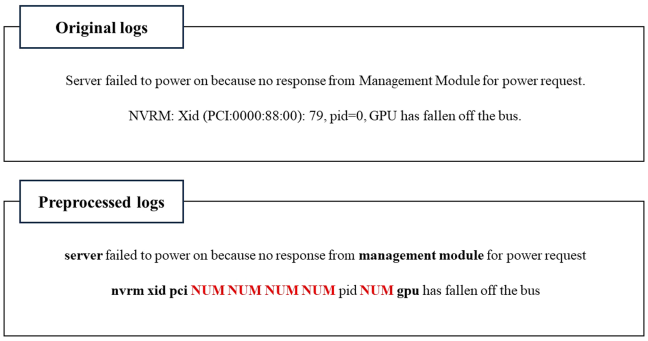


Fig. 1. Example of the preprocessed log message.

Fig. 2. Illustration of labeling process of log sequence.

happen in the future and, if an abnormal log is present, forecasts an anomaly. The model produces a log sequence that is expected to occur in the future and predicts an anomaly if an anomalous log is present in the generated sequence. This technique uses self-supervised learning of the log creation to attempt to predict errors, however, it has drawbacks due to the classification-based approach and the insufficient use of the log messages as natural language.

III. Main Method - FALL

This section presents the proposed method, FALL. By adding lexical similarity and limited vocabulary, which are traits of system log message data, into the detecting anomalies in text using ELECTRA (DATE) [30], a text-based anomaly detection approach, the FALL is created to be better suitable for early system failure identification. The proposed method involves the following three key phases: preprocessing, training, and testing.

A. Preprocessing and Windowing

In contrast to previous studies [14] and [18], which utilized log IDs in the preprocessing stage, this study does not employ log IDs, and only conducts minimal preprocessing of system log message data such as converting to lowercase, removing punctuations, and processing numbers. In particular, the numbers in the system log message data are substituted with the word "NUM" as in [11]. Through this preprocessing, the loss of information contained in the system log message can be minimized while enabling the anonymization of sensitive information within the log messages [36]. Fig. 1 shows the system log message as a result of the preprocessing. In addition, repetitive and sequential logs are eliminated during the preprocessing stage, leaving only one instance [36].

In this study, the preprocessing system of log message data is divided by node, which is a component of the HPC system, and then sorted in chronological order. Fig. 2 shows the labeling process for the sorted log message. Each log message, represented as x, consists of a total of n tokens, which can be expressed as $x = (t_1, t_2, \ldots, t_n)$. To serve as input for the model, the log sequence, S_i, is composed of 30 consecutive log messages, with $S_i = (x_i, x_{i+1}, \ldots, x_{i+29})$. In other words, S_i consists of h tokens, which represent the total number of tokens in 30 log messages, and can be expressed as $S_i = (t_i, t_{i+1}, \ldots, t_{h-1}, t_h)$. The label for the log message sequence S_i is determined based on the occurrence of failures within a defined future time interval after the lead time from the last log message x_{i+29} in the sequence. It is classified as normal if there are no failures within this period; otherwise, it is classified as abnormal. Furthermore, the sliding window technique was applied by considering the log message sequence S_i as one window, with the window stride set to 15 based on the system log message x_i. That is, when the current log sequence is S_i, the next log sequence becomes $S_{i+1} = (x_{i+15}, x_{i+16}, \ldots, x_{i+44})$.

The window size and stride were set based on the advice of industry experts. First, the window size, if too small or too large, would result in either too little or too much information used for the model's judgment. Second, the choice of stride was made to ensure the availability of log sequence data and to enhance the diversity of log message arrays within log sequences. Subsequently, industry experts reviewed the collected log sequence dataset to determine whether data should be removed. This process aims to prevent biased learning and training inefficiencies through the removal of duplicate data. Finally, the labeled log sequence dataset was only used for training with the normal data.

B. Replaced Token Detection and Replaced Mask Detection

The structure and training method of the proposed FALL is similar to the DATE, which is based on ELECTRA [31]. Fig. 3 shows the overall training process of the FALL. A discriminator and a generator are the two primary parts of the FALL.

The generator, functioning as a masked language model (MLM), substitutes the masked tokens in the input with different tokens. Here, masked tokens are those concealed by the model in a manner that obscures their meanings within the input sentence. Specifically in this study, a random generator is used as the generator, implying that masked tokens are replaced by uniformly selecting words from the vocabulary generated by the tokenizer. The discriminator performs two tasks simultaneously: the replaced mask detection (RMD) task which predicts the masking pattern applied to the input and the replaced token

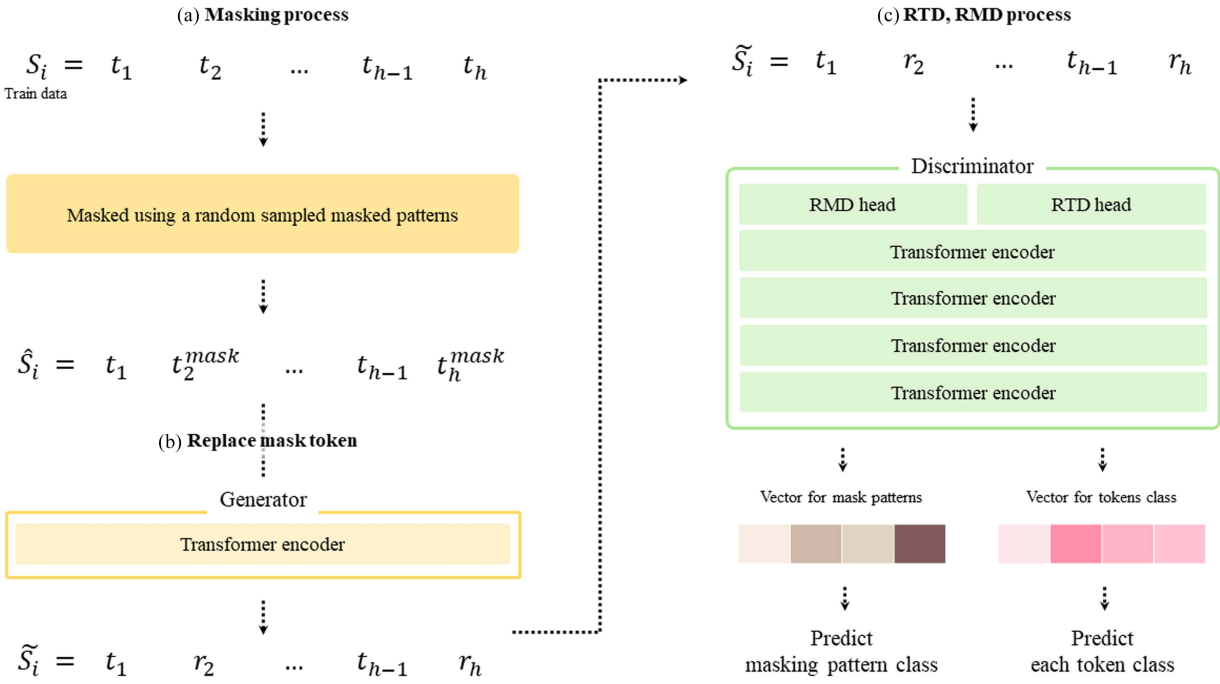


Fig. 3. A schematic overview of training phase. First, masking is applied to S_i. Second, the masked token in $\hat{S}_i$ is replaced with another word. Finally, $\tilde{S}_i$ is fed into the discriminator to carry out the RTD and RMD tasks.

detection (RTD) task which predicts whether each token in the input is a replaced or original token. Both generator and discriminator are composed of transformer encoders. While the discriminator is also utilized in the evaluation stage to detect early failures, the generator is only used for model training.

As mentioned above, the input of the FALL is a normal log sequence, S in which 30 log message data tokens are sequentially connected. The total number of tokens in S, denoted as 'h,' will either be greater or less than the model's hyperparameter, the max sequence token length. When 'h' is less than the max sequence token length, we use meaningless [PAD] tokens to fill the gap and match the token count to the max sequence token length. On the other hand, when 'h' is greater than the max sequence token length, the sentence is divided into sequences based on the max sequence token length, and they are processed sequentially.

For the input S of the model, K masking patterns, determined in advance, are randomly applied, with the masking pattern being a predefined format that determines the masking locations applied to the tokens within the input sentence. For example, assuming that m_i is one of K masking patterns and S to which m_i is applied can be expressed as $\hat{S} = (t_1, t_2, t_3^{mask}, t_4, \ldots, t_h)$. The generator receives $\hat{S}$ as an input, and the mask token inside $\hat{S}$ is replaced with another word according to the probability of the vocabulary learned by the generator. Finally, the generator produces $\tilde{S}$, which can be expressed as follows:

$$\tilde{S} = (\tilde{t_1}, \tilde{t_2}, \tilde{t_3}, \tilde{t_4}, \ldots, \tilde{t_h})$$
$$\tilde{t_i} = \begin{cases} t_i, & t_i \neq t_i^{mask} \\ r_i \sim P_G(t_i|\tilde{S} : \theta_G), & t_i = t_i^{mask} \end{cases} \quad (1)$$

The discriminator consists of a transformer encoder with RTD head and RMD head, and receives the output $\tilde{S}$ generated by the generator as its input. The RTD head is responsible for evaluating if each token in the input is an original token or a replacement token, allowing the discriminator to differentiate between log sequences based on token units. Simultaneously, the RMD head detects the masking pattern applied to the log sequence input, enabling the discriminator to learn the ability to discriminate log sequences in terms of their sequence-level characteristics. In other words, the generator and discriminator learn the contextual properties of tokens and sequences for normal log sequences. At this stage, the model learns the overall loss, comparising MLM loss, RTD loss, and RMD loss as detailed in (3), (4), and (5). This overall loss is governed by (2), with λ and μ serve as hyperparameters defining the ratio of RTD loss to RMD loss.

$$\mathcal{L}_{total} = \mathcal{L}_{MLM} + \mu\mathcal{L}_{RTD} + \lambda\mathcal{L}_{RMD} \quad (2)$$

$$\mathcal{L}_{MLM} = E\left[\sum_{\substack{i=1 \\ s.t.\, t_i \neq t_i^{mask}}}^{h} -logP_G(t_i|\tilde{S};\theta_G)\right] \quad (3)$$

$$\mathcal{L}_{RTD} = E\left[\sum_{\substack{i=1 \\ t_i \neq [CLS]}}^{h} -logP_G(t_i^{mask}|\tilde{S};\theta_D)\right] \quad (4)$$

$$\mathcal{L}_{RMD} = E[-logP_M(m = m_i|\tilde{S};\theta_D)]. \quad (5)$$

C. Anomaly Score Based on Characteristics of Log Sequence

The performance of the model is assessed through the anomaly detection task by using only the discriminator. Based

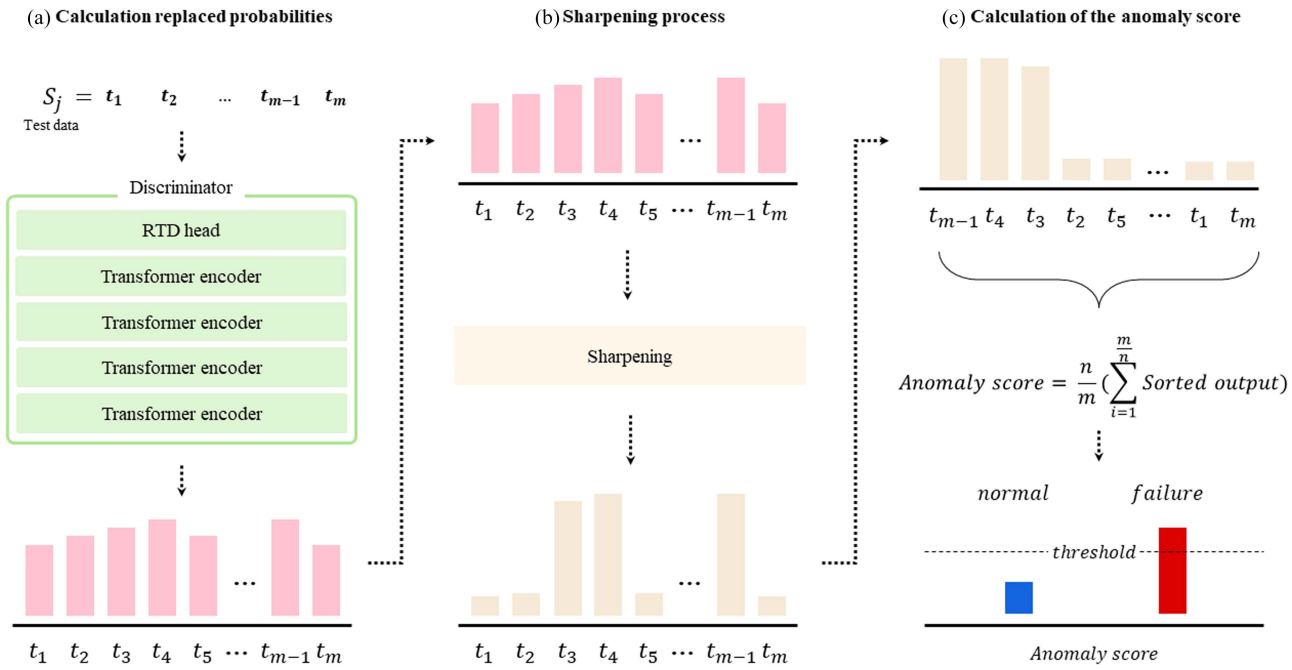


Fig. 4. A schematic overview of testing about the FALL. First, we use the trained discriminator to compute the replacement probability of the input. Second, we apply sharpening method to the calculated replacement probabilities. Finally, we generate the anomaly score by sorting the probabilities in descending order and utilizing only a portion of them.

on the likelihood that each token exiting the RTD head is a replacement token, the anomaly score is determined to decide whether it is normal or abnormal. For instance, when a normal log sequence is an input into the discriminator during evaluation, the tokens in the log sequence are tokens learned by the model, so the discriminator assigns a low probability to each token. However, because the model has not learned the abnormal log sequence, it is concluded that it has been substituted and a higher probability is provided than the token of the usual log sequence. These reasons lead to the establishment of an anomaly score that depicts the traits of the log sequence based on the likelihood that emerges from the RTD head.

In this study, the anomaly score was determined by considering the unique characteristics of log sequences in the probability calculated by the RTD head as shown in Fig. 4. The performance of early failures detection was specifically enhanced by adding the properties of log sequences into an existing text-based anomaly detection algorithm. Log sequences have two distinct characteristics: "small lexical difference between normal and abnormal log sequence" and "restricted vocabulary."

First, the lexical difference between normal and abnormal log sequences is relatively small. Natural language typically shows a wide range of vocabulary usage in certain topics such as economy, medicine, and sports. We can solve the problem of identifying abnormality in natural language-based environments by making use of lexical variations. However, in the context of log sequences, the distinctions in vocabulary are not signed because the system log pertains to a specific domain and comprises both normal and abnormal entries. In Fig. 5, we can find the normal log message 'Memory device at location arg1 is present' and the abnormal log message 'Memory device at location arg1 is overheating. These two log messages are nearly identical in all words except for 'present' and 'overheating.' In essence, the differentiation between normal and abnormal hinges on a small portion. Additionally, because system log messages in a log sequence combine normal and abnormal log messages, the lexical differences between log sequences are even less distinguishable.

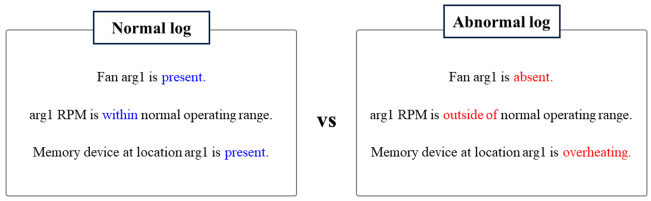


Fig. 5. Examples of slight lexical differences between normal and abnormal log sequences.

As a result, we used the sharpening method to effectively distinguish between normal and abnormal log sequences. Sharpening is a technique to regulate the difference in input values while preserving the relative ranks of each label. Sharpening can be done by (6) in which L and T represent the number of labels and a temperature that regulate the degree of difference. Tokens associated with early failure indicators are given higher probabilities by applying sharpness to the probabilities from the RTD head, while the other tokens are given lower probabilities.

$$sharpen(S,T) := \frac{t_i^{\frac{1}{T}}}{\sum_{j=1}^{L} t_j^{\frac{1}{T}}}. \tag{6}$$

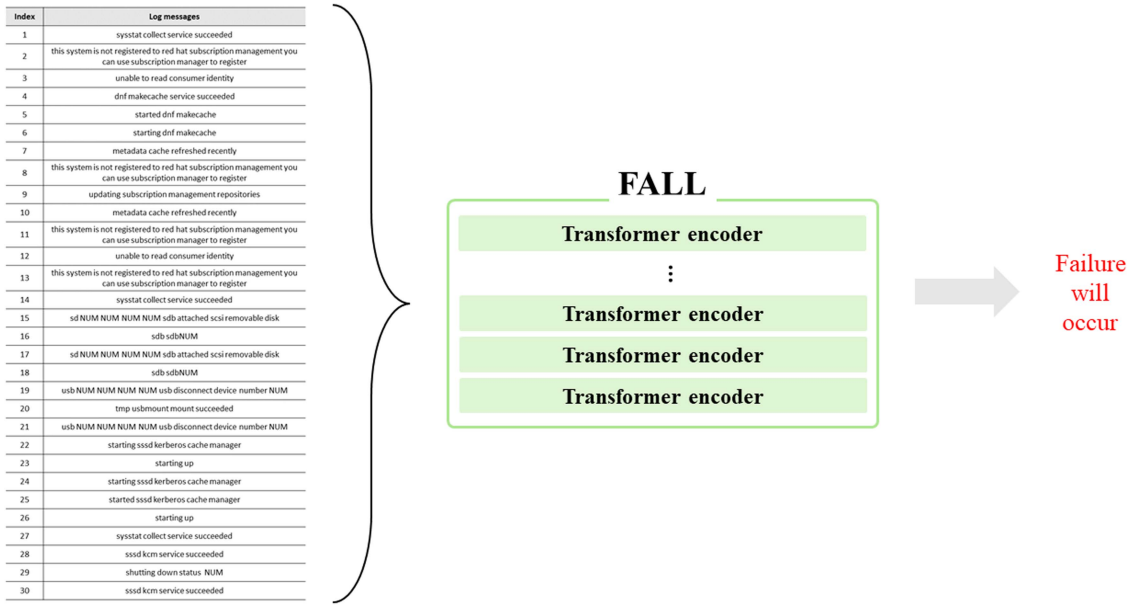


Fig. 6. The overall flow of how FALL performs anomaly detection involves taking 30 log messages and predicting the early detection of future errors.

Second, the vocabulary utilized in system log messages is restricted compared to general natural language. Sentences written by humans are included in the datasets used for natural language processing, which results in a broad and diversified vocabulary. However, because the log message is a sentence generated based on a predefined format, following the status of the HPC system, the number, and types of vocabulary are quite limited in comparison to typical natural language processing datasets. Indeed, the complete dataset of log messages we collected consisted of a total of 18,779 words. This number is significantly lower than the words commonly used in everyday language and slightly more than half of the word count used in large language models, which is 30,522. Given the limited vocabulary, the vocabulary related to pre-alert failures is also limited. For instance, consider a system log message that reads 'The storage battery for disk drive bay arg1 is absent.' In this message, the only word that serves as an indication of pre-alert and error is 'absent.'

Consequently, computing the anomaly score solely using the subset of tokens that are most likely to be alternate tokens is more effective for early failure detection than evaluating the probability of all other tokens for each token from the RTD head. Given the characteristics of these log messages, only a proportion of *1/n* tokens with a high likelihood of being replacement tokens among all tokens in the log sequence were used to calculate the anomaly score. The resulting anomaly score reflecting the two characteristics of the log sequence can be defined as follows:

$$\textit{Anomaly score} = \frac{n}{m}\left(\sum_{i=1}^{\frac{m}{n}}(reverse\ sort(Sharpen(S,T))\right). \quad (7)$$

TABLE I
DESCRIPTION ABOUT DATASET

Log data	From	To	Number of log messages
BMC log + syslog	2021-10-07	2022-07-30	33,563,534

IV. EXPERIMENTS

A. Dataset and Experiment Setup

To measure the effectiveness of the proposed FALL, which operates similarly to Fig. 6, we collected and used as a dataset the baseboard management controller (BMC) log messages and syslog messages among the system log messages of the HPC system in actual operation. The BMC is a specialized microcontroller that is integrated into many computer systems and gathers information on the physical conditions of a computer, server, or another hardware device such as temperature and humidity. Based on these data, the BMC expresses the status of the device as a log message, referred to as a BMC log message. Syslog is a log generation and management tool commonly used in Linux/Unix environments, which records all events occurring on devices as log messages [37]. Therefore, syslog messages are valuable data that allows device administrators to monitor issues, performance, and other conditions of the devices. In this experiment, the two types of system log message data were combined into a single dataset, rather than being treated as separate datasets. By evaluating both the general health of the HPC system and the physical condition of its hardware components, one can predict failures effectively. Table I shows a summary of the dataset containing over 33 million system log messages that were gathered from October 7, 2021 to July 30, 2022. The system log messages consist of approximately 3 million BMC

TABLE II
DESCRIPTION OF FIVE EXPERIMENT SCENARIOS ABOUT CLASSIFICATION MODEL

Window size	Prediction interval		Train dataset		Test dataset	
	Start	End	Total	Failure	Total	Failure
30	10	30	355,398	4,236	121,684	33,893
	30	60	356,567	2,285	106,859	18,288
	60	180	355,986	3,240	110,972	22,620
	180	300	356,231	2,827	110,971	22,620
	300	420	356,105	3,038	112,574	24,307

The window size is measured in terms of the number of log messages and lead time is quantified in seconds. The dataset is assessed based on the number of log sequences.

TABLE III
DESCRIPTION OF FIVE EXPERIMENT SCENARIOS ABOUT ANOMALY DETECTION MODEL

Window size	Prediction interval		Train dataset	Test dataset	
	Start	End	Total	Total	Failure
30	10	30	351,162	121,684	33,893
	30	60	354,282	106,859	18,288
	60	180	352,746	110,972	22,620
	180	300	353,404	110,971	22,620
	300	420	353,067	112,574	24,307

The window size is measured in terms of the number of log messages and lead time is quantified in seconds. The dataset is assessed based on the number of log sequences.

log messages and about 30 million syslog messages, collected in a ratio of 1:10.

Following the preprocessing and labeling processes outlined in Section III-A, a series of scenarios was developed by varying the lead time and prediction interval. These scenarios were then applied to the obtained log message dataset as presented in Tables II and III. Based on the current log sequence, there are a total of five scenarios that estimate the probability that an error will happen in 10 seconds, 30 seconds, 60 seconds, 180 seconds, and 300 seconds in the future.

The process of building each scenario dataset is as follows: The collected log sequence dataset is initially divided into normal and abnormal datasets. Subsequently, each dataset is split into training and test dataset while preserving the dataset's time-series structure. The normal and abnormal training data are merged into the training dataset, and the normal and abnormal test data are merged into the test dataset. Finally, both the resulting training and testing datasets are randomized using a predetermined seed number to mitigate potential bias in the model's learning process.

The training dataset used about 350,000 log sequences for each scenario out of all log sequences, and among them, about 3,000 log sequences, indicating failures, accounted for only 1% of the entire dataset. The validation dataset used approximately 110,000 log sequences for each scenario, and about 20,000 logs that are errors in the entire dataset. The training dataset was constructed to reflect the reality of having a low proportion of failures within the dataset, while the validation dataset was composed of a larger number of failures log sequences to assess the model's ability to identify a diversity of failures.

Tables II and III provide summaries of the classification model and anomaly detection model datasets, respectively. The anomaly detection model in Table III does not employ error data during training, and this aspect is duly accounted for. Except for this difference, the remaining aspects of both datasets are identical.

The proposed FALL that consists of generator and discriminator uses transformer encoders in the first layer for the generator and transformer encoders in the fourth layer for the discriminator. A word from vocabularies chosen using a uniform distribution replaces the mask token as a result of the generator's random selection. In addition, the masking pattern was set to $K = 50$, and the maximum length of the log sequence, which serves as the input of the model, was set to 128. The hyperparameter for the sharpening method, T, was set to 1/2, particularly $1/n = 1/4$ when calculating the anomaly score. Furthermore, the ratio of loss functions used for model training was set to $\mu = 50$ and $\lambda = 100$, and AdamW was used as the optimizer. A total of 20,000 training steps were performed in a batch of 128. We performed experiments by using the same evaluation protocols on three different random seeds and used the averaged performance and standard deviation.

The majority of the experiment involved two comparison tests using classification models and one comparison experiment using a text-based anomaly detection technique. The classification models including recurrent neural network (RNN), LSTM [38], gated recurrent unit (GRU) [39], and 1D convolution neural network (1D CNN) were evaluated after training using both normal and failure log sequences. On the other hand, isolation forest [40], OCSVM [41], deep support vector data description (deep SVDD) [42], and DATE [30] were used in comparative tests with anomaly detection models, and only normal log sequences were used for training.

B. Evaluation Metrics

In this study, we assess the performance of early failure detection using both the area under the receiver operating characteristic curve (AUC-ROC), a commonly used metric in the field of anomaly detection and the average number of false positives (FP) as key evaluation metrics.

For all key values, the receiver operating characteristic (ROC) curve shows the true positive rate (TPR) (8), which is the ratio of errors predicted by the model to real errors, and the false positive rate (FPR) (9), which is the ratio of mistakes falsely predicted by the model to the actual number of normal cases.

$$TPR = \frac{TP}{TP + FN} \tag{8}$$

$$FPR = \frac{FP}{FP + TN}. \tag{9}$$

The ROC curve is a graph showing the change in performance of the model about the change in the threshold value. The area

TABLE IV
COMPARISON RESULTS FOR RNN, LSTM, GRU, 1D CNN, AND FALL IN TERMS OF AVERAGED SCORES AND THEIR STANDARD DEVIATION UNDER FIVE DIFFERENT SCENARIOS BASED ON AUC SCORE

Window size	Prediction interval		Model				
	Start	End	RNN	LSTM	GRU	1D CNN	FALL (proposed)
30	10	30	0.770 (0.027)	0.671 (0.022)	0.798 (0.012)	**0.940** (0.009)	0.9241 (0.005)
	30	60	0.743 (0.001)	0.759 (0.003)	0.803 (0.002)	0.888 (0.006)	**0.9132** (0.004)
	60	180	0.793 (0.019)	0.827 (0.002)	0.829 (0.002)	0.908 (0.004)	**0.9270** (0.002)
	180	300	0.809 (0.022)	0.834 (0.005)	0.832 (0.004)	0.894 (0.001)	**0.9152** (0.002)
	300	420	0.809 (0.024)	0.833 (0.005)	0.831 (0.003)	0.906 (0.001)	**0.9246** (0.002)

Standard deviations are shown in parentheses. The window size is measured in terms of the number of log messages and lead time is quantified in seconds. The best results are in bold.

under the ROC curve is referred to as AUC and ranges from 0.5 to 1. A higher value indicates better performance of the model. In other words, it was projected that the proposed FALL would operate normally after the lead time if its anomaly score did not exceed a particular threshold and that an error would happen after the lead time if it did. Finally, after calculating TPR and FPR by changing the threshold value, the ROC curve, and AUC value were computed.

Furthermore, in this study, we used the average number of FP as the second evaluation metric. FP refers to the errors where the model incorrectly classifies actual negatives as positives. This evaluation metric is particularly significant in the field of early failure detection. The reason is that misclassifying normal situations as failure situations can lead to substantial costs. If the HPC system is predicted to experience an error in a few minutes even though it is functioning correctly, administrators will take subsequent actions accordingly. This results in unnecessary interventions incurring both temporal and financial costs. While the first evaluation metric, AUC, is commonly used in anomaly detection tasks, it still has the potential to overlook the occurrence of FP. Therefore, we adopt the average occurrence of FP as well as AUC as an evaluation metric, ensuring a more precise and comprehensive assessment.

C. Results

Table IV, shows the comparative experimental results of the proposed FALL and the classification model, revealing that the FALL achieved superior performance in all scenarios except for first one. In general, all scenarios indicate AUC performance of around 0.7 or higher for RNN-based models because they are usually well-suited for sequence data. Additionally, even though abnormal log sequences are uncommon, RNN-based models seem to have predicted failures very effectively probably because latent representations for them were trained. Furthermore, the 1D CNN model demonstrates performance above 0.9, which is higher AUC value in all scenarios than RNN-based models. The results also present that the 1D CNN model achieved superior performance in the first scenario, surpassing other models. This outcome can be attributed to the efficacy of max pooling in identifying pre-alert tokens during the 1D CNN process. However, the CNN filters, although leveraging local features of the log sequences, do not fully capture the contextual information. This results in decreased performance in the scenarios except for the first scenario. On the other hand, in the case of the FALL, we found performance above 0.91, which is a higher AUC value in all scenarios except for first scenario than other comparative models, showing that failures can be effectively predicted early. In contrast to RNN-based models, which perform worse in scenarios with shorter prediction intervals than those with longer prediction intervals, the proposed FALL exhibits a constant performance of 0.91 or higher across all prediction intervals.

Furthermore, Table V demonstrates that the proposed FALL model outperforms other classification models in three scenarios. Overall, the results in Table V exhibit notable discrepancies from those in Table IV. For instance, looking at the performance of the GRU in the first scenario, Table IV positioned it as the third best model, while in Table V, it generates over 25,000 FP. Similarly, when considering the results for the LSTM in the second scenario, Table IV ranked it as the fourth best model, whereas in Table V, it produced the lowest number of FPs, with just 235. As discussed in Section IV-B earlier, this illustrates how the actual number of FPs is concealed by AUC performance. However, the proposed FALL model consistently achieved commendable performance in both Tables IV and V. Particularly, the number of FPs generated is impressive, staying below 700 in all scenarios.

Tables VI and VII presents the comparative experimental results of the proposed FALL and the anomaly detection models. Because isolation forest, OCSVM, and deep SVDD are not anomaly detection models that are particularly suited for text data, additional embedding techniques were required. In this case, embedding was conducted in a total of three ways. The first method was the global vectors for word representation (GloVe) average approach that proceeds with embedding in word units through GloVe [43] and computes the average of each word vector. The second technique used the fastText average method which calculates the average of each word vector after embedding in word units using fastText [44]. Finally, we used InferSent [45] that conducts embedding on a sentence-by-sentence

TABLE V
COMPARISON RESULTS FOR RNN, LSTM, GRU, 1D CNN, AND FALL IN TERMS OF AVERAGED SCORES AND THEIR STANDARD DEVIATION UNDER FIVE DIFFERENT SCENARIOS BASED ON AVERAGE NUMBER OF FP

Window size	Prediction interval		Model				
	Start	End	RNN	LSTM	GRU	1D CNN	FALL (proposed)
30	10	30	3985 (3918)	821 (254)	26933 (2614)	7659 (2472)	**658** (113)
	30	60	377 (78)	**235** (17)	554 (99)	3141 (219)	637 (158)
	60	180	1019 (255)	1372 (150)	2139 (334)	4823 (948)	**691** (229)
	180	300	495 (114)	813 (219)	949 (379)	3462 (698)	**489** (102)
	300	420	**464** (125)	788 (292)	625 (80)	4091 (159)	598 (237)

Standard deviations are shown in parentheses. The window size is measured in terms of the number of log messages and lead time is quantified in seconds. The best results are in bold.

TABLE VI
COMPARISON RESULTS FOR ISOLATION FOREST, OCSVM, DEEP SVDD, DATE, AND FALL IN TERMS OF AVERAGED SCORES AND THEIR STANDARD DEVIATION UNDER FIVE DIFFERENT SCENARIOS BASED ON AUC SCORE

Window size	Prediction interval		Model				
	Start	End	OCSVM	Isolation Forest	Deep SVDD	DATE	FALL (proposed)
30	10	30	0.586 (0.001)	0.795 (0.001)	0.502 (0.08)	0.9199 (0.005)	**0.9241** (0.005)
	30	60	0.554 (0.076)	0.845 (0.005)	0.710 (0.02)	0.9057 (0.007)	**0.9132** (0.004)
	60	180	0.584 (0.059)	0.796 (0.002)	0.754 (0.02)	0.917 (0.001)	**0.9270** (0.002)
	180	300	0.648 (0.001)	0.790 (0.001)	0.730 (0.07)	0.9086 (0.001)	**0.9152** (0.002)
	300	420	0.634 (0.001)	0.811 (0.001)	0.735 (0.03)	0.9171 (0.002)	**0.9246** (0.002)

Standard deviations are shown in parentheses. The window size is measured in terms of the number of log messages and lead time is quantified in seconds. The best results are in bold.

TABLE VII
COMPARISON RESULTS FOR ISOLATION FOREST, OCSVM, DEEP SVDD, DATE, AND FALL IN TERMS OF AVERAGED SCORES AND THEIR STANDARD DEVIATION UNDER FIVE DIFFERENT SCENARIOS BASED ON AVERAGE NUMBER OF FP

Window size	Prediction interval		Model				
	Start	End	OCSVM	Isolation Forest	Deep SVDD	DATE	FALL (proposed)
30	10	30	43908 (44)	9135 (517)	33893 (16)	88165 (12)	**658** (113)
	30	60	14749 (25547)	9301 (175)	18288 (15)	88571 (19)	**637** (158)
	60	180	29412 (25472)	8784 (366)	25922 (29)	88187 (23)	**691** (229)
	180	300	44306 (53)	9797 (877)	22620 (23)	88351 (31)	**489** (102)
	300	420	44031 (236)	9137 (336)	24307 (21)	88267 (27)	**598** (237)

Standard errors are shown in parentheses. The window size is measured in terms of the number of log messages and lead time is quantified in seconds. The dataset is assessed based on the number of log sequences. The best results are in bold.

basis. We trained and assessed the three models by altering the three embedding methods, and the final performance was calculated based on the highest performance for each model.

It can be seen that the performance of the isolation forest, OCSVM, and deep SVDD models, which are optimized for an image or structured data, is lower than that of RNN-based models. In contrast, DATE and FALL, which are text-data-specific anomaly detection methods achieved 0.9 or greater in all scenarios. The proposed FALL yielded higher performances than the DATE in all scenarios, albeit not significantly. These tests show that the proposed FALL is successful for early failure detection by properly taking into account the characteristics of log sequences. Table VII follows a similar trend. In comparison to Table V, it is evident that the performance of the classification models, on the whole, lags behind. Additionally, despite DATE displaying generally high performance in Table VI, the significantly higher count of FPs can be observed. In essence, one can conclude that FALL is better suited for log data compared to DATE.

Additional experiments were conducted to confirm the impact of sharpening and the utilization of selective tokens on the FALL. Table VIII depicts the extent to which each factor influenced the proposed method by AUC performance. Consequently, both

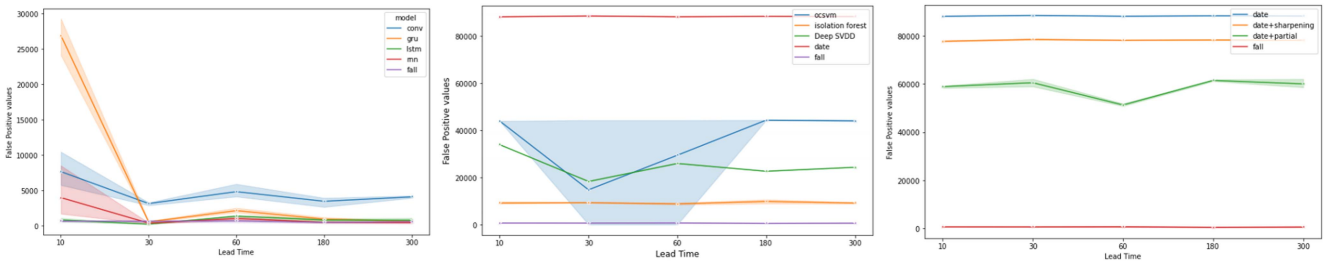


Fig. 7. Graphs depicting average and range of false positive values for each principal scenario according to method. The left graph illustrates the false positive performance results for classification models, and the middle graph represents the false positive performance results for anomaly detection models. The right graph illustrates the false positive performance results for each component of FALL. In each graph, the solid line represents the average false positive value, and the band around the line signifies the minimum and maximum values.

TABLE VIII
PERFORMANCE RESULTS FOR EACH COMPONENT OF THE FALL IN TERMS OF AVERAGED SCORES AND THEIR STANDARD DEVIATION UNDER FIVE DIFFERENT SCENARIOS BASED ON AUC SCORE

Window size	Prediction interval		Model			
	Start	End	DATE	DATE + sharpening	DATE + partial token	FALL (proposed)
30	10	30	0.9199 (0.005)	0.9206 (0.005)	0.9221 (0.005)	**0.9241** (0.005)
	30	60	0.9057 (0.007)	0.9037 (0.007)	0.9062 (0.005)	**0.9132** (0.004)
	60	180	0.917 (0.001)	0.9178 (0.001)	0.9232 (0.003)	**0.9270** (0.002)
	180	300	0.9086 (0.001)	0.9097 (0.003)	0.9143 (0.002)	**0.9152** (0.002)
	300	420	0.9171 (0.002)	0.9177 (0.002)	0.9222 (0.001)	**0.9246** (0.002)

The window size is measured in terms of the number of log messages and lead time is quantified in seconds. The dataset is assessed based on the number of log sequences. The best results are in bold.

TABLE IX
PERFORMANCE RESULTS FOR EACH COMPONENT OF THE FALL IN TERMS OF AVERAGED SCORES AND THEIR STANDARD DEVIATION UNDER FIVE DIFFERENT SCENARIOS BASED ON FP SCORE

Window size	Prediction interval		Model			
	Start	End	DATE	DATE + sharpening	DATE + partial token	FALL (proposed)
30	10	30	88165 (12)	77791 (213)	58945 (458)	**658** (113)
	30	60	88571 (19)	78571 (216)	60559 (1610)	**637** (158)
	60	180	88187 (23)	78187 (96)	51312 (449)	**691** (229)
	180	300	88351 (31)	78351 (121)	61506 (356)	**489** (102)
	300	420	88267 (27)	78267 (124)	60055 (1787)	**598** (237)

The window size is measured in terms of the number of log messages and lead time is quantified in seconds. The dataset is assessed based on the number of log sequences. The best results are in bold.

variables had an effect on AUC performance, and the FALL that used both variables had the greatest AUC value. In addition, when each element was applied individually, it can be shown that performance gradually increased in the DATE, the baseline model, except for the second scenario. Sharpening increases the lexical difference between the regular log sequence and the abnormal log sequence, allowing the model to distinguish between the two types of logs with accuracy. The experiment reveals that the AUC value on average was increased by approximately 0.0008 for each scenario. Additionally, utilizing only certain tokens reflects the characteristics of log messages with a limited vocabulary, by choosing, and utilizing tokens that are essential for early error detection. This also resulted in an increase in the AUC value by an average of 0.004 for each scenario. In the end, the FALL, which merged these two components, outperformed the baseline model DATE by boosting the AUC value by an average of roughly 0.008 for each scenario.

Furthermore, in Table IX, it is evident that the application of individual elements leads to a reduction in the number of FPs. Notably, when using only partial tokens for anomaly score, as opposed to sharpening, a substantial improvement is observed. However, when additional elements are incorporated into the existing DATE, an increase in standard deviation is noticed, indicating instability in its performance.

Fig. 7 presents graphs illustrating the mean false positive values and the range (minimum/maximum) for each principal scenario categorized by methodology. We intended to conduct performance comparisons that include not only standard deviation but also comparisons of minimum and maximum values. This approach allows for precise comparisons among models with similar false positive values. The x-axis represents the lead time in seconds, and the y-axis denotes the count of false positives. Specifically, the left graph represents the false positive performance graph for classification models. In the case of other classification models, notable performance discrepancies were observed depending on the scenario. However, for the proposed FALL model demonstrate consistent and stable performance across all scenarios. Moreover, upon inspecting the graph for anomaly detection in the center and the graphs for each component of FALL on the right, it becomes evident that FALL outperforms other models. Especially, considering the entire range (min/max), the proposed model consistently exhibited superior performance. In essence, FALL not only exhibits superior performance but also demonstrates a universal efficacy.

Lastly, additional experiments were conducted to determine if the proposed model FALL performs effectively on different datasets. The datasets used for these experiments were the Thunderbird [46] and BGL [46] datasets, which are benchmark datasets [47] in the field (https://github.com/logpai/loghub). Both datasets consist of log messages collected from high-performance computing (HPC) systems, aligning well with the scenarios assumed in this paper. Tables X and XI present the evaluations of DATE and FALL based on the Thunderbird and BGL datasets, respectively. In the Thunderbird dataset, it is evident that FALL outperforms DATE in terms of AUC in all scenarios except the second one. Particularly noteworthy is the significantly lower average number of FP achieved by FALL.

TABLE X
PERFORMANCE COMPARISON BETWEEN DATE AND FALL BASED ON THE THUNDERBIRD DATASET, USING AUC AND THE AVERAGE NUMBER OF FP AS PERFORMANCE METRICS

Window size	Prediction interval		AUC		average of FP	
	Start	End	DATE	FALL	DATE	FALL
30	10	30	0.770 (0.003)	**0.778** (0.002)	9267 (111)	**81** (11)
	30	60	**0.760** (0.001)	0.720 (0.002)	9258 (98)	**55** (10)
	60	180	0.749 (0.002)	**0.753** (0.005)	9263 (52)	**34** (9)
	180	300	0.875 (0.003)	**0.881** (0.003)	9262 (77)	**23** (7)
	300	420	0.842 (0.002)	**0.847** (0.003)	9254 (102)	**14** (5)

Standard errors are shown in parentheses. The window size is measured in terms of the number of log messages and lead time is quantified in seconds. The best results are in bold.

TABLE XI
PERFORMANCE COMPARISON BETWEEN DATE AND FALL BASED ON THE BGL DATASET, USING AUC AND THE AVERAGE NUMBER OF FP AS PERFORMANCE METRICS

Window size	Prediction interval		AUC		average of FP	
	Start	End	DATE	FALL	DATE	FALL
30	10	30	0.864 (0.001)	**0.882** (0.003)	4225 (111)	**105** (21)
	30	60	**0.791** (0.002)	0.767 (0.002)	4661 (98)	**26** (11)
	60	180	0.785 (0.005)	**0.791** (0.002)	4683 (52)	**34** (12)
	180	300	0.812 (0.004)	0.812 (0.003)	4352 (77)	**104** (17)
	300	420	0.810 (0.001)	**0.811** (0.001)	4436 (102)	**23** (7)

Standard errors are shown in parentheses. The window size is measured in terms of the number of log messages and lead time is quantified in seconds. The best results are in bold.

This advantage of FALL is equally evident in the BGL dataset. As observed in Table XI, FALL exhibits superior performance compared to DATE in the first, third, and fifth scenarios, accompanied by consistently lower average FP counts in all scenarios. Thus, FALL demonstrates its robust performance across different log data sources.

V. CONCLUSION

This study presents a text-based anomaly detection method, FALL, for the early detection of failures in HPC systems. For the first time in the field, the FALL creates tasks for anomaly detection using log sequences as natural language. To address the problem of misclassification caused by the small vocabulary difference between normal and error log sequences, the proposed method utilizes a sharpening. Furthermore, by focusing on the features of limited vocabulary, anomaly detection performance is improved by using only some tokens that are essential for early failure detection. Empirical findings show that the FALL enhances early failure detection by utilizing the characteristics of log sequences. Experiments demonstrate that the FALL outperforms other methods, achieving an average AUC value of more than 0.91.

While the FALL showed excellent performance in early failure detection of HPC systems, there are opportunities for future research to enhance the method. Preprocessing log messages with domain-specific tokens appropriate for system logs is one such area for development. Currently, only numbers are replaced with a token called NUM, which may lead to loss of information. It is expected that the model would be able to acquire more predictive information and perform better by tokenizing specific aspects like IP addresses or file locations within the log messages.

Furthermore, research into the number of node failures and the correlation analysis between them would be highly meaningful. In this paper, we focus solely on predicting the future occurrence of node failures through the model. For example, in [48], research was conducted on examining the correlation between failures when one or more node failures occurred. Based on this research, conducting a post-analysis on the correlation of failures after early detection of node failures could lead to even more significant research.

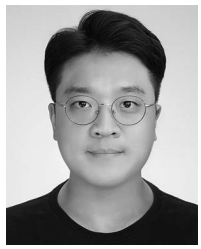
Finally, research on predicting specific errors among system failures such as CPU errors, DRAM errors, etc., using a natural language prediction model is a plausible avenue. In this paper, we designed a model optimized for predicting node failures and demonstrated its effectiveness. However, we did not explore research on specific error types that lead to node failures. Investigating specific errors poses additional challenges, requiring more extensive data and potentially encountering imbalanced data, which could impede effective learning by natural language models. Addressing these challenges with natural language models has the potential to enhance the efficiency of error prediction within HPC systems, with the added benefit of simplified maintenance for the natural language prediction model.

REFERENCES

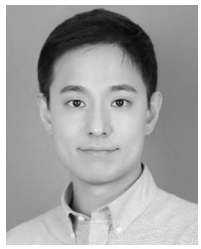
[1] S. Almeida, "An introduction to high performance computing," *Int. J. Modern Phys. A*, vol. 28, no. 22/23, 2013, Art. no. 1340021.
[2] W. Fan, F. Xiao, J. Fan, Z. Han, L. Sun, and R. Wang, "Fault-tolerant routing with load balancing in LeTQ networks," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 68–82, Jan./Feb. 2023.
[3] D. Xiang, B. Li, and Y. Fu, "Fault-tolerant adaptive routing in dragonfly networks," *IEEE Trans. Dependable Secure Comput.*, vol. 16, no. 2, pp. 259–271, Mar./Apr. 2019.
[4] M. Schulz, B. Lucas, T. Macaluso, D. J. Quinlan, and J. Wu, "Inter-agency workshop on HPC resilience at extreme scale," *Nat. Secur. Agency Adv. Comput. Syst.*, Feb. 2012. [Online]. Available: https://api.semanticscholar.org/CorpusID:15095322
[5] F. Cappello, G. Al, W. Gropp, S. Kale, B. Kramer, and M. Snir, "Toward exascale resilience: 2014 update," *Supercomput. Front. Innovations: Int. J.*, vol. 1, no. 1, pp. 5–28, 2014.
[6] M. Rodríguez-Pascual, J. Cao, J. A. Moríñigo, G. Cooperman, and R. Mayo-Garc ía, "Job migration in HPC clusters by means of checkpoint/restart," *J. Supercomput.*, vol. 75, pp. 6517–6541, 2019.
[7] N. El-Sayed and B. Schroeder, "Understanding practical tradeoffs in HPC checkpoint-scheduling policies," *IEEE Trans. Dependable Secure Comput.*, vol. 15, no. 2, pp. 336–350, Mar./Apr. 2018.
[8] F. Cappello, "Fault tolerance in petascale/exascale systems: Current knowledge, challenges and research opportunities," *Int. J. High Perform. Comput. Appl.*, vol. 23, no. 3, pp. 212–226, 2009.
[9] E. N. Elnozahy and J. S. Plank, "Checkpointing for peta-scale systems: A look into the future of practical rollback-recovery," *IEEE Trans. Dependable Secure Comput.*, vol. 1, no. 2, pp. 97–108, Second Quarter 2004.
[10] A. Gainaru, F. Cappello, M. Snir, and W. Kramer, "Failure prediction for HPC systems and applications: Current situation and open issues," *Int. J. High Perform. Comput. Appl.*, vol. 27, no. 3, pp. 273–282, 2013.
[11] Y. Lee, J. Kim, and P. Kang, "LAnoBERT: System log anomaly detection based on BERT masked language model," 2021, *arXiv:2111.09564*.
[12] K. A. Alharthi, A. Jhumka, S. Di, and F. Cappello, "Clairvoyant: A log-based transformer-decoder for failure prediction in large-scale systems," in *Proc. 36th ACM Int. Conf. Supercomput.*, 2022, pp. 1–14.

[13] K. Zhang, J. Xu, M. R. Min, G. Jiang, K. Pelechrinis, and H. Zhang, "Automated it system failure prediction: A deep learning approach," in *Proc. IEEE Int. Conf. Big Data*, 2016, pp. 1291–1300.
[14] A. Das, F. Mueller, C. Siegel, and A. Vishnu, "Desh: Deep learning for system health prediction of lead times to failure in HPC," in *Proc. 27th Int. Symp. High- Perform. Parallel Distrib. Comput.*, 2018, pp. 40–51.
[15] S. Huang et al., "HitAnomaly: Hierarchical transformers for anomaly detection in system log," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 4, pp. 2064–2076, Dec. 2020.
[16] S. Nedelkoski, J. Bogatinovski, A. Acker, J. Cardoso, and O. Kao, "Self-attentive classification-based anomaly detection in unstructured logs," in *Proc. IEEE Int. Conf. Data Mining*, 2020, pp. 1196–1201.
[17] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "SwissLog: Robust anomaly detection and localization for interleaved unstructured logs," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 2762–2780, Jul./Aug. 2023.
[18] A. Das, F. Mueller, and B. Rountree, "Aarohi: Making real-time node failure prediction feasible," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2020, pp. 1092–1101.
[19] J. R. Campos, E. Costa, and M. Vieira, "Online failure prediction for complex systems: Methodology and case studies," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 3520–3534, Jul./Aug. 2023.
[20] Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," in *Proc. 38th Int. Conf. Softw. Eng. Companion*, 2016, pp. 102–111.
[21] Z. Lan, Z. Zheng, and Y. Li, "Toward automated anomaly identification in large-scale systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 21, no. 2, pp. 174–187, Feb. 2010.
[22] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Comput. Secur.*, vol. 28, no. 1/2, pp. 18–28, 2009.
[23] C. C. Aggarwal, *Outlier Analysis Second Edition*. Springer Int. Publishing, 2016. [Online]. Available: https://books.google.co.kr/books?id=KyG1DQAAQBAJ
[24] L. M. Manevitz and M. Yousef, "One-class SVMs for document classification," *J. Mach. Learn. Res.*, vol. 2, pp. 139–154, 2001.
[25] A. Mahapatra, N. Srivastava, and J. Srivastava, "Contextual anomaly detection in text data," *Algorithms*, vol. 5, no. 4, pp. 469–489, 2012.
[26] R. Kannan, H. Woo, C. C. Aggarwal, and H. Park, "Outlier detection for text data: An extended version," 2017, *arXiv: 1701.01325*.
[27] L. Bontemps, V. L. Cao, J. McDermott, and N.-A. Le-Khac, "Collective anomaly detection based on long short-term memory recurrent neural networks," in *Proc. 3rd Int. Conf. Future Data Secur. Eng.*, Can Tho City, Vietnam, Springer, 2016, pp. 141–152.
[28] L. Manevitz and M. Yousef, "One-class document classification via neural networks," *Neurocomputing*, vol. 70, no. 7/9, pp. 1466–1481, 2007.
[29] L. Ruff, Y. Zemlyanskiy, R. Vandermeulen, T. Schnake, and M. Kloft, "Self-attentive, multi-context one-class classification for unsupervised anomaly detection on text," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 4061–4071.
[30] A. Manolache, F. Brad, and E. Burceanu, "DATE: Detecting anomalies in text via self-supervision of transformers," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2021, pp. 267–277.
[31] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, "ELECTRA: Pre-training text encoders as discriminators rather than generators," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–18. [Online]. Available: https://openreview.net/pdf?id=r1xMH1BtvB
[32] K. Lang, "NewsWeeder: Learning to filter netnews," in *Proc. 12th Int. Conf. Mach. Learn.*, A. Prieditis and S. Russell, Eds. San Francisco CA: Morgan Kaufmann, 1995, pp. 331–339.
[33] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 649–657.
[34] C. A. Rincón, J.-F. Pâris, R. Vilalta, A. M. Cheng, and D. D. Long, "Disk failure prediction in heterogeneous environments," in *Proc. Int. Symp. Perform. Eval. Comput. Telecommun. Syst.*, 2017, pp. 1–7.
[35] E. W. Fulp, G. A. Fink, and J. N. Haack, "Predicting computer system failures using support vector machines," in *Proc. 1st USENIX Conf. Anal. Syst. Logs*, 2008, Art. no. 5.
[36] S. Ghiasvand and F. M. Ciorba, "Assessing data usefulness for failure analysis in anonymized system logs," in *Proc. 17th Int. Symp. Parallel Distrib. Comput.*, 2018, pp. 164–171.
[37] R. Gerhards, "The syslog protocol," RFC, Tech. Rep., vol. 5424, pp. 1–38, Mar. 2009. [Online]. Available: https://www.rfc-editor.org/info/rfc5424
[38] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
[39] J. Chung, Ç. Gülçehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*. [Online]. Available: http://arxiv.org/abs/1412.3555
[40] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, 2008, pp. 413–422.
[41] B. Schölkopf, R. C. Williamson, A. Smola, J. Shawe-Taylor, and J. Platt, "Support vector method for novelty detection," in *Proc. Adv. Neural Inf. Process. Syst.*, 1999, pp. 582–588.
[42] L. Ruff et al., "Deep one-class classification," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 4393–4402.
[43] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2014, pp. 1532–1543.
[44] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *Trans. Assoc. Comput. Linguistics*, vol. 5, pp. 135–146, 2017.
[45] A. Conneau, D. Kiela, H. Schwenk, L. Barrault, and A. Bordes, "Supervised learning of universal sentence representations from natural language inference data," 2017, *arXiv: 1705.02364*. [Online]. Available: http://arxiv.org/abs/1705.02364
[46] A. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *Proc. 37th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2007, pp. 575–584.
[47] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," 2020, *arXiv: 2008.06448*.
[48] S. Ghiasvand, F. M. Ciorba, R. Tschüter, and W. E. Nagel, "Lessons learned from spatial and temporal correlation of node failures in high performance computers," in *Proc. 24th Euromicro Int. Conf. Parallel Distrib. Netw.-Based Process.*, 2016, pp. 377–381.

Jaeyoon Jeong received the BS degree in applied statistics from Konkuk University, Seoul, South Korea, in 2021. He is currently working toward the MS degree with the School of Industrial and Management Engineering, Korea University, Seoul, South Korea. His research interests include anomaly detection and their industrial applications.



Insung Baek received the BS degree from Korea University, Seoul, Korea, in 2016, where he is currently working toward the PhD degree in industrial and management engineering. His research interests include explainable artificial intelligence algorithms and their application to games and visions.



Byungwoo Bang received the BS degree in computer engineering from Dongguk University, in 2007, and the MS degree in software engineering from the Graduate School of the Department of Computer Science and Engineering, Korea University, Seoul, South Korea, in 2014. He is currently working toward the PhD degree on the topic of RAS with the Department of Industrial Management, Korea University, while working with Samsung Advanced Institute of Technology, Suwon, South Korea. He is currently the leader of the RAS team. His research interests include detecting supercomputer's failure using various techniques in high performance computing systems.

Junyeon Lee received the BS degree in human environment and design from Yonsei University, in 2013, and the MS degree in software from the Graduate School of the Department of Computer Science and Engineering, Korea University, Seoul, South Korea, in 2021. He is currently a staff researcher with the Samsung Advanced Institute of Technology, Suwon, South Korea. His research interests include improving reliability and availability in high performance computing systems.

Uiseok Song received the BS degree in computer science and the MS degree in distributed computing from the Department of Computer Science and Engineering, Sogang University, Seoul, South Korea, in 2016 and 2018, respectively. He is currently a researcher with the Samsung Advanced Institute of Technology, Suwon, South Korea. His research interests include AI and fault analysis for systems and devices.

Seoung Bum Kim received the MS degree in industrial and systems engineering, and the PhD degree in industrial and systems engineering from the Georgia Institute of Technology, in 2001 and 2005, respectively. He is a professor with the School of Industrial and Management Engineering, Korea University. From 2005–2009, he was an assistant professor with the Department of Industrial and Manufacturing Systems Engineering, University of Texas at Arlington. His research interests utilize machine learning algorithms to create new methods for various problems appearing in engineering and science. He has published more than 150 internationally recognized journals and refereed conference proceedings.